

Aula 35: Projeto Reativo Full-Stack - Configuração Inicial e Backend Base

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1h)

1. Arquitetura do Projeto

Componentes Principais

Camada	Tecnologias	Responsabilidade
Frontend	Vue 3 + Quasar	Interface do chat e dashboard
Backend	Spring WebFlux + WebSockets	Lógica real-time e API REST
Banco de Dados	PostgreSQL (R2DBC)	Armazenamento de mensagens
Infraestrutura	Docker	Containerização do banco

Fluxo de Comunicação



Parte Prática (3h)

Atividade 1: Setup do Backend (1h30min)

Passo 1: Inicialização do Projeto

bash

```
# Criar projeto Spring Boot
spring init --dependencies=webflux,websocket,data-r2dbc,postgresql reactive-chat-backend
```

Passo 2: Configuração do PostgreSQL (Docker)

```
bash
```

```
docker run --name chat-db -e POSTGRES_PASSWORD=secret -p 5432:5432 -d postgres:14
```

Passo 3: Entidade e Repository

```
java
```

```
// Message.java
public record Message(
    @Id UUID id,
    String content,
    String sender,
    Instant timestamp
) {}

// MessageRepository.java
public interface MessageRepository extends ReactiveCrudRepository<Message, UUID> {}
```

Passo 4: Controller REST Básico

```
java
```

```
@RestController
@RequestMapping("/api/messages")
@RequiredArgsConstructor
public class MessageController {
    private final MessageRepository repo;

    @GetMapping
    public Flux<Message> getAll() {
        return repo.findAll();
    }
}
```

Atividade 2: Setup do Frontend (1h30min)

Passo 1: Inicialização com Quasar CLI

```
bash
```

```
npm init quasar@latest chat-frontend  
# Escolher: Vue 3 + TypeScript + Quasar
```

Passo 2: Instalação do STOMP.js

```
bash
```

```
npm install @stomp/stompjs
```

Passo 3: Componente Básico de Chat

```
vue
```

```
<!-- ChatInput.vue -->  
<template>  
  <q-input v-model="message" @keyup.enter="send" />  
</template>  
  
<script setup lang="ts">  
import { ref } from 'vue';  
const message = ref('');  
const send = () => console.log('Mensagem enviada:', message.value);  
</script>
```

Tarefa para Casa

1. **Backend:** Implementar endpoint `POST /api/messages` com validação.
2. **Frontend:** Criar componente `MessageList.vue` para exibir mensagens estáticas.

Próxima Aula

- **Tópico:** Implementação de WebSockets no backend.
- **Atividade prática:** Configurar STOMP e enviar mensagens entre clientes.

Checklist de Resultados

```
plaintext
```

- ✓ Backend Spring Boot configurado com R2DBC
- ✓ Banco PostgreSQL rodando via Docker
- ✓ Frontend Quasar inicializado
- ✓ Componente ChatInput.vue criado

https://miro.medium.com/v2/resize:fit:1000/1*HXGZhRkG0Lf8YlrGJ1Hl4A.png (*Visão geral da arquitetura*) 🚀

Dica: Use `spring-boot-devtools` para reinicialização automática durante o desenvolvimento!